Everything you call – transitively – from a coroutine should be non-blocking.

Coroutines are contagious to callees.

Everything that calls – transitively – to a coroutine must iterate the generator.

Coroutines are contagious to callers.

Coroutines are contagious to callees.

Coroutines are contagious to callers.

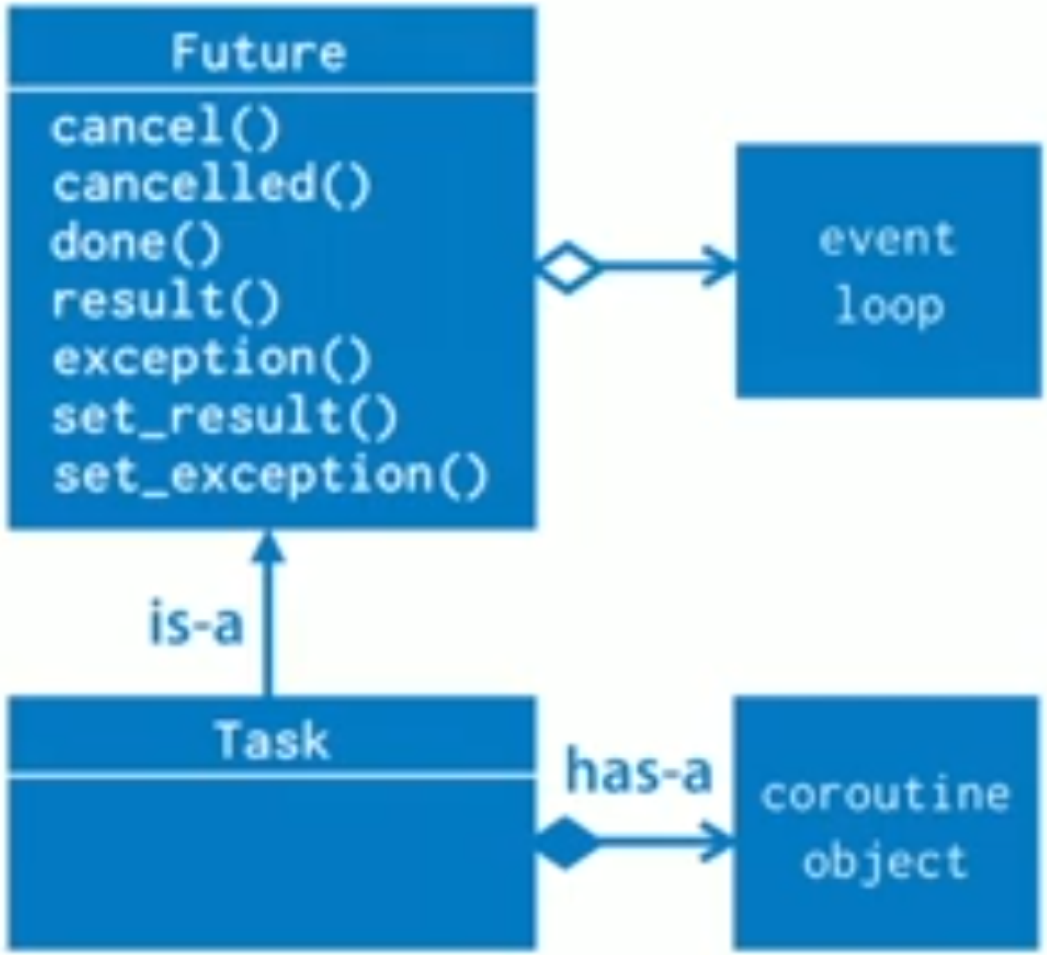
asyncio

- Coroutines implement tasks
- Coroutines await other coroutines
- Event-loop schedules concurrent tasks
- Tasks must not block
- Awaiting facilitates context switches
- To yield control without needing a result `await asyncio.sleep(0)`

Task

A subclass of Future which wraps a coroutine

```
async def thirteen_digit_prime(x):
    return (await is_prime(x)) and len(str(x)) == 13


async def monitor_future(future, interval_seconds):
    while not future.done():
        print("Waiting...")
        await asyncio.sleep(interval_seconds)
    print("Done!")


loop = asyncio.get_event_loop()
future = loop.create_future()

coro_obj = search(lucas(),
                  thirteen_digit_prime)

search_task = loop.create_task(coro_obj)

loop.create_task(monitor_future(search_task, 1.0))
loop.run_until_complete(search_task)
print(search_task.result())
loop.close()
```



Coroutines versus Coroutine Objects

Try to be precise when naming or documenting

Coroutine

```
async def search(iterable, predicate):
    for item in iterable:
        if predicate(item):
            return item
        await asyncio.sleep(0)
    raise ValueError("Not found")
```

code

callable

Coroutine Object

```
>>> c = search(lucas(), is_prime)
>>> c
<coroutine object search at 0x109a1e1a8>
```

code + execution state

awaitable

Event loops

The AbstractEventLoop has a large interface

```
run_forever()
run_until_complete(future)
is_running()
stop()
is_closed()
close()
shutdown_asyncgens()
call_soon(callback, *args)
call_soon_threadsafe(callable, *args)
call_later(delay, callable, *args)
call_at(when, callable, *args)
time()
create_future()
create_task(coroutine_object)
set_task_factory(callable)
get_task_factory()
```

start and stop the event-loop

scheduling callbacks

factories you shouldn't use

configuration you rarely need

Event loops

The AbstractEventLoop has a large interface

```
set_exception_handler(handler)
get_exception_handler()
default_exception_handler(context)
call_exception_handler(context)
get_debug()
set_debug(enabled)
add_signal_handler()
remove_signal_handler()
run_in_executor(executor, callable, *args)
set_default_executor(executor, callable, *args)
```

exception handling

diagnostics

signal handling

run blocking code in other threads or processes

Event loops

The AbstractEventLoop has a large interface

```
sock_recv(sock, nbytes)
sock_sendall(sock, data)
sock_connect(sock, address)
sock_accept(sock)
getaddrinfo(host, port)
getnameinfo(sockaddr)
```

low-level socket operations

Event loops

The AbstractEventLoop has a large interface

```
create_connection(protocol_factory, …)
create_datagram_endpoint(protocol_factory, …)
create_unix_connection(protocol_factory, …)
create_server(protocol_factory, …)
create_unix_server(protocol_factory, …)
connect_accepted_socket(…)
add_reader(fd, callback)
remove_reader(fd)
add_writer(fd)
remove_writer(fd)
connect_read_pipe(fd)
connect_write_pipe(fd)
```

protocol-based SSL/TLS/TCP/UDP socket servers

watch file descriptors

connect to pipes

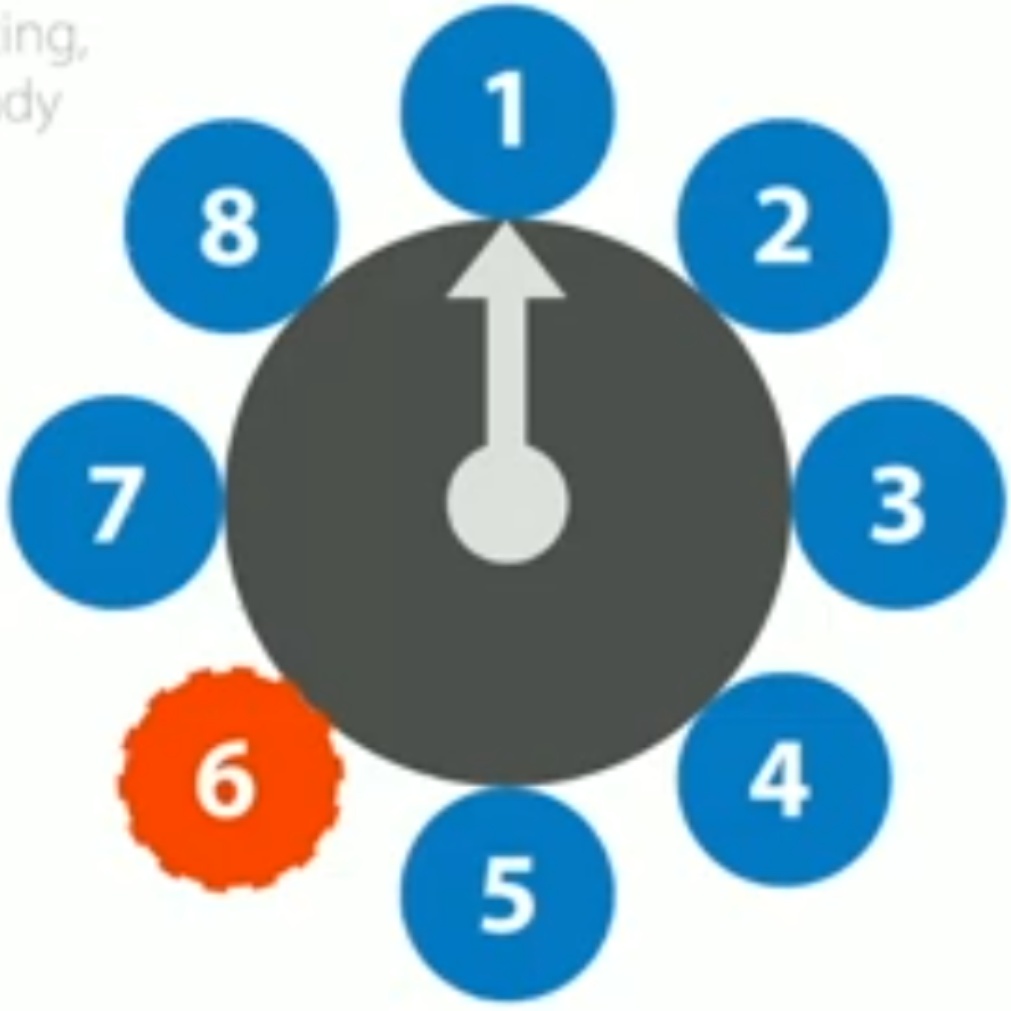
I/O aware scheduler

Tasks suspend when waiting,
scheduled when data ready

waiting
on I/O

I/O ready

scheduled



Layered abstractions for asyncio

Progressively higher-level, from coroutines … to coroutines.